SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

Laboratory Project Presentation

Cloud Storage Technologies

Object, Block, and File Storage Compared

A Comparative Study

Presented by: Jaivant(192311326)

Guided By: Sampath Kumar . K

ABSTRACT

Why storage architecture matters, and what this study compares

- Cloud storage underpins nearly all modern applications, and choosing the right storage architecture directly affects application performance, cost, and scalability.
- This project compares the three core cloud storage models — block, file, and object storage — examining how each organizes, addresses, and exposes data to applications.
- The study evaluates access patterns, architectural trade-offs, and real-world fit for each model, mapping representative workloads (databases, shared file systems, media/backup) to the storage type built for them.
- Results show there is no single universally-best storage type; the right choice depends on matching the storage architecture to the application's access pattern, scale, and latency needs.

INTRODUCTION

- Global data generation is growing exponentially, driving rapid adoption of cloud storage across enterprises, applications, and personal devices.
- Different workloads need fundamentally different access patterns: raw disk-level speed, shared hierarchical files, or web-scale API access to billions of objects.
- Cloud platforms expose three core storage models — block, file, and object — each with a distinct way of organizing and addressing data.
- This project studies each model's architecture, then maps it to the real-world workload it was built to serve, before comparing trade-offs across all three.

Block + File + Object =
Unified Cloud Data Platform

Modern applications typically combine more than one model rather than relying on a single storage type.

CORE TECHNICAL EXPLANATION

Three ways cloud platforms organize and expose data

Block Storage

Data is split into fixed-size blocks, each with a unique address, and mounted like a raw disk. The OS/application controls the file system on top.

Low-latency, high-performance disk

File Storage

Data is stored as files inside a hierarchical folder structure and accessed over a shared network protocol (NFS/SMB), just like a traditional file server.

Shared, hierarchical access

Object Storage

Data is stored as self-contained objects (data + metadata + unique ID) in a flat namespace, accessed via HTTP/REST APIs at massive scale.

Flat, metadata-rich, web-scale

RESEARCH GAP

What existing storage comparisons still don't cover

1. Most existing material documents block, file, and object storage separately, with limited side-by-side technical comparison in one place.

2. Quantitative comparison of latency, throughput, and scalability across all three models on equivalent workloads is rarely presented together.

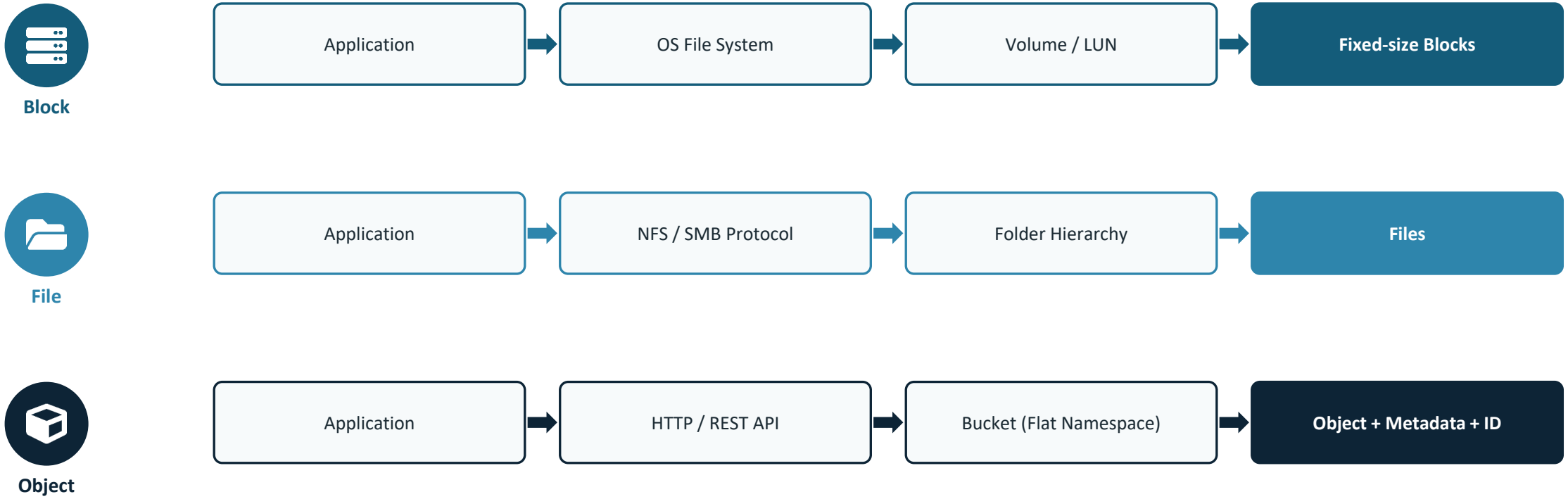
3. The real cost and complexity of migrating data between storage models is often missing from architecture guides and vendor documentation.

4. Limited attention is given to how modern applications combine more than one storage model rather than choosing just one.

This project addresses these gaps through a structured technical comparison and architecture-level analysis of all three storage models.

ARCHITECTURE / PROCESS DIAGRAM

How an application actually reaches its data in each model



Block trades flexibility for raw speed · File trades scale for familiar shared access · Object trades fine-grained edits for massive, durable scale.

ADVANTAGES, DISADVANTAGES & APPLICATIONS

Advantages

- Matching storage type to workload cuts both latency and cost versus a one-size-fits-all approach
- Elastic, pay-as-you-grow capacity with no physical hardware to manage
- High built-in durability and availability across all three cloud-native models
- Object and file models scale to shared, multi-client access with minimal setup

Disadvantages

- Migrating data between block, file, and object models is costly and often needs re-architecting
- Running multiple storage models adds operational and monitoring complexity
- Object storage's higher latency limits it for transactional, low-latency workloads
- Cost can become unpredictable at scale without careful tiering and lifecycle policies

Applications

- Block: production databases and boot volumes needing raw disk speed
- File: shared team workspaces and applications needing a common folder structure
- Object: media, backups, and data-lake storage at massive scale
- Most real systems combine two or three models across their stack

Thank You

Questions & Discussion Welcome